



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-47162 Vim: Vimsript Code Injection in netrw NetrwBookHistSave() via crafted directory name

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-13
Version	1.0
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-47162 Vim: Vimsript Code Injection in netrw NetrwBookHistSave() via crafted directory name - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-13 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

1. Executive Summary

CVE-2026-47162 is a Vim netrw code-injection vulnerability that turns a crafted directory name into attacker-controlled Vimscript the next time the victim's netrw history file is sourced. The flaw sits in ``s:NetrwBookHistSave()`` inside ``runtime/pack/dist/opt/netrw/autoload/netrw.vim``: vulnerable builds serialize browsed directory paths into `~/vim/.netrwhist`` by embedding the raw directory name inside a single-quoted Vimscript string. If the directory name itself contains a single quote and additional Vimscript tokens, the saved history file becomes executable payload.

This is not an internet-facing wormable service bug. Exploitation is user-assisted and filesystem-mediated: a victim must browse a maliciously named directory with netrw, or otherwise cause that path to be written into netrw history, and later initialize netrw again so the history file is sourced. That constraint materially lowers opportunistic mass-exploitation risk, but it still creates a credible local code-execution path on developer workstations, shared jump hosts, CI helpers, analysis VMs, and multi-user Linux systems where Vim is routinely used against untrusted repos, archives, or extracted file trees.

Public severity normalization is inconsistent. GitHub marks the upstream advisory Moderate, while downstream enrichment tracked by Tenable and NVD-style ecosystems scores the issue High (CVSS v3.1 8.8; CVSS v4.0 7.3). The technical reality sits between those views: the bug can lead to arbitrary Vimscript and shell command execution in the victim's context, but exploitation still depends on user interaction and a crafted filesystem path rather than a clean remote network trigger.

Local validation on this analyst host materially increased confidence in the defensive assessment. The installed Ubuntu package ``vim 2:9.1.2141-1ubuntu4.3`` reproduced the vulnerable behavior using a safe proof-of-concept adapted from Vim's own regression test pattern: a crafted directory-history value wrote an unsafe line into ``.netrwhist``, and sourcing that file executed the injected statement. Immediate defender priorities are to upgrade or backport-patch Vim/netrw, inventory endpoints where Vim is present, and hunt for suspicious shell execution from Vim plus anomalous ``.netrwhist`` content.

2. Intelligence Requirement

This report addresses the following intelligence requirement: what does CVE-2026-47162 mean for defenders using Vim, which workflows are actually exposed, how urgent is remediation relative to the user-assisted trigger model, and what detection and hunting actions should be prioritized now?

Question	Assessment
What is the flaw?	A Vimscript code-injection vulnerability in netrw history serialization allows crafted directory names to break out of a single-quoted string written to <code>~/vim/.netrwhist`</code> .
What is the primary impact?	Arbitrary Vimscript execution, including shell command execution via <code>`system()`</code> or <code>`:!`</code> , in the security context of the user launching Vim/netrw.
Which component is affected?	Vim's bundled netrw plugin, specifically <code>`s:NetrwBookHistSave()`</code> in <code>`autoload/netrw.vim`</code> .
Which versions are affected?	Vim releases before <code>`9.2.0495`</code> , with downstream distro packages remaining vulnerable unless they have explicitly backported the fix.
What fixes the issue?	Upstream patch <code>`9.2.0495`</code> , which switches vulnerable history serialization to <code>`string()`</code> -based quoting.
Is this an internet-facing RCE?	No. Exploitation is user-assisted and depends on interaction with a crafted directory path plus later sourcing of netrw history.
Is exploitation broadly confirmed?	Reviewed public sources did not provide authoritative evidence of broad in-the-wild exploitation, CISA KEV inclusion, or mass-scanning activity as of 2026-06-15.

Why should defenders still care?	Vim is widely deployed on Linux, Unix, and security workstations, and the vulnerability can turn routine browsing of untrusted extracted content into local code execution.
Which environments deserve highest priority?	Shared Linux systems, developer workstations, analysis VMs, bastions, and automation nodes where Vim is used against untrusted archives, repos, or user-controlled file paths.

3. Source Review and Confidence

Source	Contribution	Confidence
Microsoft Security Update Guide syndication entry	Intake trigger confirming public publication visibility and canonical CVE label	Medium
GitHub Security Advisory `GHSa-crm5-rh6j-2c7c`	Canonical upstream description, affected-version statement, advisory severity, and CWE mapping	High
Vim patch commit `f08ab2f4d7d2947c8dd6c179ae08ee6146a2694b`	Ground-truth engineering details showing the vulnerable line and the remediation using `string()`	High
Vim release `v9.2.0495`	Confirms the fix shipped in an upstream release milestone	High
Debian Security Tracker	Downstream distro exposure context showing several package branches still marked vulnerable while sid is fixed	High
Tenable CVE enrichment	Public CVSS, reference normalization, CISA KEV absence, and additional ecosystem metadata	Medium
Local analyst validation on Ubuntu `vim 2:9.1.2141-1ubuntu4.3`	Real execution evidence that a current downstream package on this host still reproduces the unsafe `.netrw` sourcing path	High

Confidence is high for the core technical assessment because the upstream advisory, patch diff, release tag, and local proof-of-concept all align on the root cause: unsafely serializing a filesystem-derived directory name into executable Vimscript. Confidence is medium for exploitation-prevalence judgments because the reviewed public record focuses on disclosure and patching rather than incident-response evidence, observed campaigns, or authoritative victim telemetry.

A second confidence caveat is severity normalization. GitHub's upstream advisory marks the issue Moderate, while downstream CVSS scoring surfaces High severity. For defender decision-making, the most useful framing is neither score in isolation but the actual attack preconditions: user-assisted local execution through untrusted path handling, with meaningful impact on systems where Vim is common and filesystem content is not fully trusted.

4.1 Vulnerability metadata

Field	Value
CVE	CVE-2026-47162
Product family	Vim / netrw
Affected component	`runtime/pack/dist/opt/netrw/autoload/netrw.vim`

	<code>`s:NetrwBookHistSave()`</code>
Vulnerability class	Vimscript code injection via crafted directory-name serialization
Weaknesses	CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component; CWE-94 Improper Control of Generation of Code
Upstream severity	Moderate (GitHub advisory)
Public scoring	CVSS v3.1 8.8 High; CVSS v4.0 7.3 High in downstream enrichment
Affected versions	Vim before <code>`9.2.0495`</code>
Fixed version	Vim <code>`9.2.0495`</code> or downstream builds that backport the same fix
Default history path	<code>~/vim/.netrwhist`</code>
Execution outcome	Arbitrary Vimscript execution, potentially escalating to shell command execution through <code>`system()`</code> or <code>`:!`</code>
Exploitation status	No authoritative broad in-the-wild exploitation evidence identified during this review

4.2 Flaw mechanics

The vulnerable code path writes each browsed directory path into netrw's history file as a line resembling:

```
let g:netrw_dirhist_1='...directory name...'
```

In vulnerable builds, the directory name is concatenated directly into that single-quoted string without escaping embedded single quotes. A crafted directory such as ``x'|let g:injected = 1|let y='z`` therefore produces a syntactically valid history line that closes the original string, appends attacker-controlled Vimscript, and reopens the string to preserve parsing. The malicious content is not executed at write time; execution occurs later when Vim or netrw sources ``.netrwhist`` during initialization.

The upstream fix is both small and decisive. Patch ``9.2.0495`` replaces vulnerable concatenation with ``string(g:netrw_dirhist_{cnt})``, causing the history value to be serialized with safe Vimscript quoting rules instead of raw interpolation. The associated regression test intentionally uses an injected history value and asserts that sourcing the saved file must not create the attacker-controlled variable.

4.3 Exposure profile

Exposure pattern	Why it matters	Priority
Developer or admin workstation using Vim to browse extracted archives, repos, or user-supplied directories	A socially engineered or staged path name can be recorded into netrw history and later executed in the user's context	Immediate
Shared Linux jump hosts or bastions with many operators	Vim is common, home directories persist, and local code execution can become a lateral-movement foothold	Immediate
Malware-analysis or reverse-engineering VMs handling hostile file trees	Analysts routinely inspect attacker-controlled artifacts and may browse unpacked directories with Vim/netrw	Immediate
CI helpers, automation nodes, or scripts that invoke Vim in plugin-enabled modes	Less common than interactive use, but vulnerable if netrw initialization and user-	High

	controlled paths overlap	
Minimal systems without Vim, or workflows using editors that do not source netrw history	Removes or sharply reduces the relevant attack surface	Reduced
Hardened Vim builds with confirmed backported fix or upstream `9.2.0495+`	The vulnerable serialization path is remediated	Reduced

4.4 Local validation findings

A safe local proof-of-concept was executed on this analyst host to validate whether downstream packaging still exposed the issue. The installed package reported:

Validation item	Result
Host package	`vim 2:9.1.2141-1ubuntu4.3`
Installed runtime path	`/usr/share/vim/vim91/pack/dist/opt/netrw/autoload/netrw.vim`
Vulnerable serialization line present	Yes; the local runtime still uses raw single-quote concatenation rather than `string()`
PoC method	Adapted from Vim upstream regression-test logic using a crafted history value and a temporary `.netrwhist` in the task workspace
Result	`g:injected=1` after sourcing the saved history file, confirming code injection on this host
Safety note	The proof-of-concept executed only a benign variable assignment, not an external shell command

This validation matters because it demonstrates a practical defender pitfall: version-number intuition alone can be misleading across distro packaging. A non-`9.2.x` downstream build can still be vulnerable if the specific netrw fix was not backported.

4.5 MITRE ATT&CK framing

ATT&CK mapping for CVE-2026-47162 should be treated as defensive framing rather than proof of a known campaign pattern.

ATT&CK ID	Technique	Rationale
T1059	Command and Scripting Interpreter	Successful exploitation yields attacker-controlled Vimscript and can invoke shell commands from the editor context.
T1204	User Execution	The vulnerable path normally requires a user to interact with a crafted directory and later trigger netrw history loading.

5. Threat Context

CVE-2026-47162 is best understood as a trust-boundary failure between filesystem content and executable editor state. The bug does not require a memory-corruption exploit chain or an exposed service; it abuses the fact that a directory name, normally treated as inert data, is promoted into a persisted Vimscript statement. That makes the vulnerability operationally closer to a local configuration-poisoning or state-poisoning issue than to a classic network daemon RCE.

The most realistic abuse paths are social-engineering and staging scenarios. An attacker can distribute a repository, archive, sample set, or mounted file share containing a specially named directory. A victim using netrw to inspect that path records it

into ``.netrw``; on a later Vim/netrw initialization, the payload executes. Security teams, developers, incident responders, and administrators are all plausible targets because they are more likely than average users to use terminal editors against untrusted paths.

Scenario	Description	Defensive meaning
Malicious archive or repo sent to an analyst or developer	A crafted directory name survives extraction and is browsed with netrw, poisoning persistent history	Treat filesystem path names as hostile input in analysis and engineering workflows
Shared bastion or admin host	A privileged operator triggers the issue while browsing untrusted data on a long-lived account	Prioritize patching on systems with broad credential or infrastructure access
Research or malware-analysis VM	Hostile samples are unpacked and browsed frequently, increasing exposure to unusual path strings	Analysis tooling and editor packages deserve the same patch discipline as network services
Developer workstation with plugin-enabled Vim	User-assisted local execution can lead to credential theft, shell access, or source-code tampering under the developer identity	Patch endpoints and monitor for Vim spawning shells unexpectedly

Reviewed public sources did not provide evidence of KEV inclusion, public exploit-kit packaging, or broad campaign adoption as of 2026-06-15. That lowers concern for indiscriminate mass exploitation, but it should not reduce urgency on high-trust workstations and shared Unix environments. The attack preconditions are realistic enough for targeted abuse, red-team simulation, and opportunistic compromise in environments that handle untrusted directories every day.

6. Detection and Hunting

No stable IOC set emerged from the reviewed public sources. Detection should therefore focus on vulnerable-asset discovery, anomalous ``.netrw`` activity, suspicious Vim child-process execution, and filesystem patterns suggesting path-name staging rather than on hash- or IP-based matching.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal	Caveat	
Vim packages below a fixed or backported build	Package inventory, SBOM, configuration management, EDR software inventory	Identify assets still carrying the vulnerable netrw code path	Distro backports require validation beyond major/minor version labels	
Writes to <code>`.netrw`</code> followed by later Vim initialization	File events plus process telemetry	Surface poisoning of persistent netrw history and possible trigger windows	Many environments do not capture user-home file events deeply	
Shell execution from <code>`.vim`</code> , <code>`.gvim`</code> , or <code>`.vi`</code> parents	EDR or Sysmon-style process creation telemetry	Catch likely exploit outcomes or suspicious post-editor command execution	Legitimate editor shell-outs create false positives and require tuning	
Suspicious strings inside <code>`.netrw`</code> such as <code>`</code>	<code>`</code> , <code>`.system(`</code> , <code>`.!`</code> , or unmatched quotes	File-content inspection, DFIR triage, EDR content search, scripted host checks	Confirm whether history files contain obviously weaponized lines	Content-level telemetry is uncommon outside DFIR or scripted checks

Untrusted extracted content containing directory names with quote/pipe metacharacters	File-event telemetry, sandbox triage, analyst scripts	Identify likely staging artifacts before a user opens them in Vim	Directory-name visibility varies across platforms and sensors	
---	---	---	---	--

6.2 Sigma - Vim spawning an unexpected shell

```

title: Suspicious Shell Execution from Vim or Vi Process
id: 8e2c4f20-e70d-44c1-a228-56ce5aa49d1e
status: experimental
description: Detects shell execution launched from Vim-family processes, which can indicate exploitation of CVE-2026-47162 or other editor-driven code execution paths.
author: Lost Boys Cyber
date: 2026-06-15
references:
  - https://github.com/vim/vim/security/advisories/GHSA-crm5-rh6j-2c7c
  - https://github.com/vim/vim/commit/f08ab2f4d7d2947c8dd6c179ae08ee6146a2694b
logsource:
  category: process_creation
  product: windows
detection:
  parent_editor:
    ParentImage|endswith:
      - '\\vim.exe'
      - '\\gvim.exe'
      - '\\vi.exe'
  child_shell:
    Image|endswith:
      - '\\cmd.exe'
      - '\\powershell.exe'
      - '\\pwsh.exe'
      - '\\bash.exe'
      - '\\sh.exe'
  suspicious_args:
    CommandLine|contains:
      - 'system('
      - '.netrw'
      - '-c '
      - '/c '
  condition: parent_editor and child_shell
falsepositives:
  - Legitimate use of :! or terminal shell escapes by administrators and developers
  - Build or automation scripts that intentionally invoke Vim as part of an editor-based workflow
level: high
tags:
  - attack.t1059
  - attack.t1204

```

Linux estates should tune parent-process paths for `vim`, `vi`, and wrapper binaries captured by the local EDR platform.

6.3 KQL - hunt for Vim child-shell execution

```

let lookback = 30d;
DeviceProcessEvents
| where Timestamp > ago(lookback)
| where InitiatingProcessFileName in~ ("vim", "vim.exe", "gvim", "gvim.exe", "vi", "vi.exe")
| where FileName in~ ("bash", "bash.exe", "sh", "sh.exe", "cmd.exe", "powershell.exe", "pwsh.exe")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine, InitiatingProcessFileName, InitiatingProcessCommandLine
| order by Timestamp desc

```

6.4 KQL - inventory likely-vulnerable Vim deployments and `netrw` activity

```

let lookback = 30d;
let suspectVim = DeviceTvmSoftwareInventory

```



```
| where SoftwareName has_any ("Vim", "vim")
| project DeviceId, DeviceName, SoftwareName, SoftwareVersion;
let netrwHistoryWrites = DeviceFileEvents
| where Timestamp > ago(lookback)
| where FileName == ".netrwhist"
| project Timestamp, DeviceId, DeviceName, FolderPath, InitiatingProcessFileName,
InitiatingProcessCommandLine;
suspectVim
| join kind=leftouter netrwHistoryWrites on DeviceId
| project DeviceName, SoftwareName, SoftwareVersion, Timestamp, FolderPath, InitiatingProcessFileName,
InitiatingProcessCommandLine
| order by DeviceName asc, Timestamp desc
```

6.5 Splunk hunt - Unix Vim spawning a shell

```
index=edr OR index=sysmon
(process_name=bash OR process_name=sh OR process_name=cmd.exe OR process_name=powershell.exe OR
process_name=pwsh.exe)
(parent_process_name=vim OR parent_process_name=gvim OR parent_process_name=vi)
| stats count min(_time) as firstTime max(_time) as lastTime by host user parent_process_name
process_name process command_line parent_command_line
| convert ctime(firstTime) ctime(lastTime)
| sort - lastTime
```

6.6 Threat-hunting hypotheses

Hypothesis	What to prove or disprove
Some endpoints still carry vulnerable Vim/netrw builds despite routine package updates	Correlate software inventory, distro-package versions, and host-level netrw code snippets or package advisories
`.netrwhist` files on analyst and admin systems may already contain poisoned entries	Perform targeted triage on high-value Linux and Unix home directories, looking for metacharacters, `system(`, `!`, or malformed quoting inside `.netrwhist`
Successful exploitation would appear as shell execution launched from a Vim-family parent process	Hunt EDR process trees for shells or suspicious child processes spawned by `vim`, `gvim`, or `vi`
File-discovery workflows may stage malicious directory names before code execution occurs	Review archives, extracted malware/sample folders, and shared research directories for unusual quote- and pipe-heavy path names

7. Recommendations

Priority	Owner	Action	Rationale
Immediate	Endpoint / platform engineering	Patch Vim to upstream `9.2.0495` or a verified backported build across Linux, Unix, and analyst workstations	Removes the vulnerable history-serialization path
Immediate	Security operations	Inventory hosts with Vim installed, especially bastions, analyst VMs, shared admin systems, and developer endpoints	The issue is most dangerous on high-trust interactive systems
Immediate	DFIR / threat hunting	Triage `.netrwhist` on priority systems for suspicious metacharacters, `system(`, `!`, or malformed quoting	Detects history poisoning that may precede or indicate exploitation
Immediate	Detection engineering	Deploy or adapt Vim-child-	Shell spawning from Vim is the

		shell analytics in EDR/SIEM tooling	clearest practical post-exploitation signal
High	User enablement / engineering leads	Warn power users not to browse untrusted extracted directories with vulnerable Vim builds until patching is complete	Reduces the user-assisted trigger path during remediation lag
High	Platform owners	Validate distro backport status rather than assuming a package is fixed because the OS is current	Local validation showed a current Ubuntu package still reproducing the issue
Medium	Security architecture	Consider whether high-risk workflows should use disposable analysis users or ephemeral VMs when handling hostile file trees	Limits impact if a local editor-based execution path is triggered

8. References

1. Microsoft Security Update Guide entry for CVE-2026-47162: <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-47162>
2. Vim GitHub Security Advisory `GHSA-crm5-rh6j-2c7c`: <https://github.com/vim/vim/security/advisories/GHSA-crm5-rh6j-2c7c>
3. Vim fix commit `f08ab2f4d7d2947c8dd6c179ae08ee6146a2694b`: <https://github.com/vim/vim/commit/f08ab2f4d7d2947c8dd6c179ae08ee6146a2694b>
4. Vim release `v9.2.0495`: <https://github.com/vim/vim/releases/tag/v9.2.0495>
5. Debian Security Tracker for CVE-2026-47162: <https://security-tracker.debian.org/tracker/CVE-2026-47162>
6. Tenable CVE record for CVE-2026-47162: <https://www.tenable.com/cve/CVE-2026-47162>
7. oss-security republication of the Vim advisory: <https://seclists.org/oss-sec/2026/q2/566>
8. Local analyst validation performed on 2026-06-15 against Ubuntu package `vim 2:9.1.2141-1ubuntu4.3` using a benign proof-of-concept adapted from the upstream regression-test pattern.